Cognitive State Compilation: Resource-Bounded Construction of Reasoning States for Long-Horizon LLM Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Long-horizon LLM agents draw on heterogeneous resources: retrieved knowledge, past experience, and intermediate computation, but existing systems decide what to store, retrieve, or discard largely per component, without a unified account of how these resources should be jointly transformed into the reasoning state handed to the model at each step. We formalize this as Cognitive State Compilation (CSC), a task-conditioned, resource-bounded construction problem posed as a 0/1 knapsack over a candidate pool spanning knowledge, experience, and computation resources. Our prototype scores candidates with a Gradient Boosted Tree classifier over a 9-dimensional feature vector, trained with 5-fold cross-validation (98.6% accuracy, 94.2% F1), and selects greedily by utility density under a strict token budget. At a 500-token budget on a 15-task HotpotQA multi-hop sample, CSC reaches 100% state coverage versus 83.3% for standard RAG and 60.0% for full-context and MEM1-style baselines; with a live LLM this translates to 88.9% F1 versus 66.7% for the strongest baseline. On a multi-turn long-horizon comparison, CSC maintains 100% state retention at every tested horizon up to $N = 16$, exceeding the reproduced MEM1 reference by 6–18 percentage points without any reinforcement-learning overhead. We report two limitations openly: the GBT scorer is evaluated on the same coverage signal it is trained on, so reported accuracy reflects in-distribution performance rather than held-out generalization, and our WebShop condition covers only five curated tasks. We position CSC as the explicit, budgeted, cross-resource-type state-construction problem, distinct from OS-style memory management and from MEM1's compact recurrent state.

1 Introduction

Long-horizon LLM agents increasingly operate over heterogeneous prior resources: documents and facts retrieved from external corpora, records of past experience such as successful trajectories or episodic reflections, and computational artifacts such as tool outputs, scratchpad reasoning, or cached intermediate results. As an agent accumulates history and access to larger resource pools, the cost of naively including everything grows without bound, while the token budget available for any single reasoning step remains fixed by the model's context window and inference-cost constraints. We refer to this as the token budget crisis in long-horizon agents: the gap between how much potentially relevant material an agent has accumulated and how much it can actually afford to place in front of the model for a given decision.

The dominant response to this crisis has been to optimize individual components of the memory pipeline: what to store (episodic buffers, vector databases), what to retrieve (top-k similarity search), or what to update or discard (memory-editing policies). These are valuable but distinct optimization

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

targets, and each is typically defined over a single, homogeneous resource type, most commonly a memory bank of notes or a document corpus. There is comparatively little work that treats the construction of the reasoning state itself, the object actually placed in the model's context, as a first-class, resource-bounded optimization problem over a heterogeneous pool of knowledge, experience, and computation.

We introduce Cognitive State Compilation (CSC), which formulates this construction problem explicitly as a constrained 0/1 knapsack: given a task τ, a heterogeneous candidate pool $R = R_{\text{know}} \cup R_{\text{exp}} \cup R_{\text{comp}}$, and a strict token budget B, select the subset $S \subseteq R$ that maximizes total task-conditioned utility subject to $\sum_{r \in S} \text{cost}(r) \leq B$. Our prototype scores candidates with a trained Gradient Boosted Tree classifier and selects greedily by utility density, and we evaluate it against five baselines on HotpotQA, Natural Questions, a chained multi-turn benchmark, and WebShop, plus a long-horizon comparison against MEM1.

Our contributions are:

- **Formulation.** We pose the construction of the transient reasoning state as a resource-bounded, cross-resource-type knapsack problem, unifying knowledge, experience, and computation under a single budgeted objective rather than optimizing each resource type independently.
- **Learned scorer and reference architecture.** We implement a modular pipeline in which a trained 9-feature GBT utility scorer (with a hand-specified dense scorer as a fallback when no trained model is available) feeds a greedy or exact knapsack selector, evaluated against five baselines under matched budgets.
- **Empirical validation with disclosed limitations.** We report state-coverage and live-LLM gains on HotpotQA, cross-domain coverage on three further domains, and a long-horizon comparison against MEM1, while disclosing rather than downplaying the training-label leakage in the GBT scorer and the small size of the WebShop evaluation.

The remainder of the paper is organized as follows. Section 2 situates CSC within existing long-horizon agent memory, retrieval, and tool-use literature. Section 3 formalizes the knapsack-based compilation problem together with the trained scorer, fallback scorer, and selection algorithm. Sections 4 and 5 describe the experimental setup and results across HotpotQA, cross-domain, live-LLM, and long-horizon conditions. Section 6 discusses what the evaluation does and does not show and summarizes limitations, and Section 7 concludes.

2 Related Work

This section situates CSC relative to prior work on long-horizon agent memory, retrieval-augmented generation, resource-bounded reasoning, and general LLM agent frameworks, summarized concisely here since Table 1 provides a detailed feature-by-feature comparison.

Long-horizon agent memory architectures. Recent agent memory systems each address a different piece of the storage and retrieval problem. MemGPT manages context as an OS-style virtual memory hierarchy paged by the LLM itself [Packer et al., 2023]; A-MEM organizes episodic notes in a linked, Zettelkasten-style structure [Xu et al., 2025]; Memory-R1 and AgeMem learn reinforcement-learning policies over memory-editing or memory-tier actions [Yan et al., 2025, Yu et al., 2026]; and MemOS proposes a full memory lifecycle under an operating-system framing [Li et al., 2025]. Each of these treats memory as a largely homogeneous resource and does not pose an explicit, budgeted selection problem jointly over knowledge, experience, and computation; Table 1 summarizes these differences in detail.

Agentic and multi-hop retrieval. Retrieval-augmented generation [Lewis et al., 2020] and its agentic, multi-hop extensions optimize what to retrieve from a knowledge corpus, commonly evaluated on datasets such as HotpotQA [Yang et al., 2018] and Natural Questions [Kwiatkowski et al., 2019]. These methods generally treat the retrieved pool as the sole resource type and do not jointly budget knowledge alongside experience and computational artifacts.

Resource-bounded reasoning. MEM1 is the most closely related prior system: it learns, via reinforcement learning, to consolidate memory and reasoning into a single compact recurrent state [Zhou et al., 2025]. This makes MEM1 implicitly budget-aware, but it does not explicitly select or

Table 1: Positioning of CSC relative to prior long-horizon agent memory systems. "Cross-resource state construction" denotes explicit, joint selection across knowledge, experience, and computation under a single budget objective.

System	Store	Retrieve	Update /Manage	Compute reuse	Budget aware	Cross-resource	Main limitation vs. CSC
MemGPT	✓	✓	partial	×	×	×	LLM-paged OS-style memory; no joint budgeted construction.
A-MEM	✓	✓	✓	×	partial	×	Dynamic linking of one memory type only.
Memory-R1	✓	✓	✓	×	×	×	RL-learned editing; no resource-budgeted compiler.
AgeMem	✓	✓	✓	×	partial	partial	Unifies LT/ST memory tiers only.
MemOS	✓	✓	✓	✓	×	partial	Systems-level lifecycle, no learned budget-utility tradeoff.
MEM1	✓	n/a	✓	partial	✓	✓	Recurrent RL consolidation of one state, not explicit pool selection.
CSC (ours)	✓	✓	✓	✓	✓	✓	—

budget across distinct heterogeneous resource pools using an interpretable utility-and-cost objective. CSC instead poses this construction as an inference-time knapsack problem, using a learned scorer to select and budget across knowledge, experience, and computation without additional reinforcement-learning training.

LLM agents and tool use. A broader line of work equips LLMs with the ability to reason, act, and use tools over multiple steps, including ReAct-style interleaved reasoning and acting [Yao et al., 2023], self-taught tool use [Schick et al., 2023], verbal self-reflection for iterative improvement [Shinn et al., 2023], and simulated agents with persistent memory streams [Park et al., 2023]. These frameworks motivate the setting we study but generally leave open how the material placed in context at each step should be selected under an explicit budget, which is the problem CSC addresses directly.

Existing systems generally optimize what to store, retrieve, or discard for one resource type at a time, or consolidate memory implicitly through a learned recurrent policy. CSC instead formulates the construction of the transient reasoning state itself as a unified, explicitly budgeted selection problem over knowledge, experience, and computation, using a learned utility scorer rather than hand-tuned heuristics, and we evaluate it directly against representative baselines drawn from each of these related lines of work.

3 Problem Formulation and Method

This section formalizes the reasoning-state construction problem as a resource-bounded knapsack, and describes the five components of our reference architecture: the heterogeneous candidate pool, the learned utility scorer, the fallback heuristic scorer, the resource-constrained knapsack selector, and state composition.

3.1 Heterogeneous Resource Pool

We treat the reasoning state supplied to an LLM agent at a given decision point as constructed from a candidate pool $R = R_{\text{know}} \cup R_{\text{exp}} \cup R_{\text{comp}}$, where R_{know} contains retrieved documents, facts, or passages; R_{exp} contains few-shot trajectories, successful execution histories, episodic reflections,

or prior strategy notes; and R_{comp} contains intermediate scratchpads, tool-execution traces, entity-extraction results, or attribute-filtering results. Each resource $r \in R$ carries a resource type, a text payload, a token cost $\text{cost}(r)$, and an optional utility hint $h(r) \in [0, 1]$.

3.2 Cognitive State Compilation as a Knapsack Problem

Given a task specification τ, context C, and a strict token budget B, CSC selects the subset $S \subseteq R$ solving

$$\max_{S \subseteq R} \sum_{r \in S} u(r \mid \tau, C) \quad \text{subject to} \quad \sum_{r \in S} \text{cost}(r) \leq B, \tag{1}$$

a 0/1 knapsack over heterogeneous items with task-conditioned values. Token cost is measured with a standard subword tokenizer, with a whitespace-approximation fallback where an exact tokenizer is unavailable.

3.3 Production Scorer: Trained GBT Utility Scorer

The production scorer is a Gradient Boosted Tree classifier [Chen and Guestrin, 2016] that predicts $P(\text{resource useful} \mid \tau, r)$ from a 9-dimensional feature vector combining unigram and bigram TF-IDF similarity [Pedregosa et al., 2011], keyword overlap, one-hot resource-type indicators, the raw utility hint, and a cost-normalized density feature. The classifier is trained with 5-fold stratified cross-validation on pooled data from all four benchmark domains, using $\text{useful}(r) = \mathbb{1}[r.\text{text contains a required fact of } \tau]$ as the training label, the same rule used by our evaluation metric (Section 4). Cross-validated performance is $98.6\% \pm 0.3\%$ accuracy and $94.2\% \pm 1.1\%$ F1 (precision $\approx 97\%$, recall $\approx 92\%$). We flag this explicitly as a limitation, not a strength: because the training label and the evaluation coverage metric derive from the same required-facts signal, these cross-validation numbers measure the classifier's ability to recover an already-known label rule rather than out-of-distribution generalization to a disjoint task distribution (Section 6).

3.4 Fallback Scorer: Dense Heuristic Scorer

When no trained model is available, the pipeline falls back to a hand-specified dense scorer combining bigram similarity, entity-alignment overlap, a resource-type prior, and the utility hint, weighted by $\alpha = 0.25$ and $\gamma = 0.35$, with type priors of 0.05, 0.15, and 0.20 for knowledge, experience, and computation resources respectively. A relevance gate suppresses the type and hint terms when base similarity falls below a small threshold, to avoid injecting low-relevance resources purely on type or hint priors.

3.5 Resource-Constrained Knapsack Selector

The resource-bounded selection problem formulated in Eq. (1) is an NP-hard 0/1 knapsack problem with task-conditioned item utilities [Martello and Toth, 1990]. We implement a greedy utility-density approximation selector following the classic value-to-cost density principle [Dantzig, 1957]. The selector ranks candidate resources in descending order of utility density $\rho(r) = u(r \mid \tau)/\max(1, \text{cost}(r))$ and greedily packs items into state S while enforcing the total budget constraint $\sum_{r \in S} \text{cost}(r) \leq B$ and optional per-type caps c_t, operating in $O(N \log N)$ time. The complete compilation procedure is formalized in Algorithm 1. For small candidate pools requiring provable optimality, our architecture also incorporates an exact dynamic-programming knapsack selector (detailed in Appendix C).

3.6 State Composition

Selected resources are assembled into a single structured text block, grouped and tagged by resource type and identifier, prefixed by the task query, following the COMPOSE step of Algorithm 1. The resulting object is the compiled cognitive state S, passed directly to the LLM.

Taken together, these five components define a modular pipeline in which scoring, selection, and composition are cleanly separated: the scorer can be swapped between the trained classifier and the heuristic fallback without touching the selector or composer, and the selector can be swapped

Algorithm 1 Cognitive State Compilation (CSC) [Martello and Toth, 1990]

Require: Task τ, candidate pool $R = R_{\text{know}} \cup R_{\text{exp}} \cup R_{\text{comp}}$, token budget B, utility scorer u_θ [Chen and Guestrin, 2016], optional type caps $\{c_t\}$
Ensure: Compiled cognitive state S^*

$scored \leftarrow \emptyset$
for $r \in R$ **do**
  $u(r) \leftarrow u_\theta(f(r, \tau))$ ▷ Compute utility from 9-D feature vector
  $scored \leftarrow scored \cup \{(r, u(r))\}$
end for
Sort $scored$ by utility density $\rho(r) = u(r)/\max(1, \text{cost}(r))$ descending [Dantzig, 1957]
$S^* \leftarrow \emptyset$; $remaining \leftarrow B$; $counts[t] \leftarrow 0\ \forall t \in \{\text{know}, \text{exp}, \text{comp}\}$
for $(r, u(r)) \in scored$ **do**
  if $\text{cost}(r) \leq remaining$ **and** $(c_{r.\text{type}} = \infty$ **or** $counts[r.\text{type}] < c_{r.\text{type}})$ **then**
    $S^* \leftarrow S^* \cup \{r\}$
    $remaining \leftarrow remaining - \text{cost}(r)$
    $counts[r.\text{type}] \leftarrow counts[r.\text{type}] + 1$
  end if
end for
return Compose(S^*, τ) ▷ Serialize selected resources into structured state

between the greedy and exact variants without touching the scorer. Section 4 describes how we evaluate this pipeline against representative baselines.

4 Experimental Setup

This section describes the datasets, baseline methods, metrics, and models used to assess CSC, establishing the common conditions under which all six methods are compared in Section 5.

Datasets. We evaluate on four task distributions: HotpotQA multi-hop reasoning [Yang et al., 2018], evaluated here on a 15-task sample; Natural Questions [Kwiatkowski et al., 2019]; a chained multi-turn (Q4) dialogue benchmark; and WebShop interactive e-commerce tasks [Yao et al., 2022], of which only five curated tasks are used, too few to support a generalization claim, and we do not make one.

Baselines. B0 Full Context appends resources chronologically until the budget is exhausted, with no relevance ranking. B1 Standard RAG retrieves top-k resources from the knowledge pool only via TF-IDF cosine similarity. B2 Memory-Augmented RAG extends B1 to the union of knowledge and experience pools. B3 Relevance-Only Selection applies unweighted cosine relevance across all three pools, with no type priors and no learning. B4 CSC (Ours) is the trained-GBT-scorer pipeline of Section 3. B5 MEM1-Style Recurrent is a controlled sequential fill-and-replace approximation of MEM1-style recurrent consolidation [Zhou et al., 2025]; it approximates MEM1's behavior at inference time rather than reproducing its full reinforcement-learning training procedure.

Metrics and Evaluation Models. For each compiled state S and task τ with required gold facts F_τ, we evaluate:

$$\text{State Coverage: } \text{Cov}(S, \tau) = \frac{1}{|F_\tau|} \sum_{f \in F_\tau} \mathbb{1}[f \in \text{text}(S)], \tag{2}$$

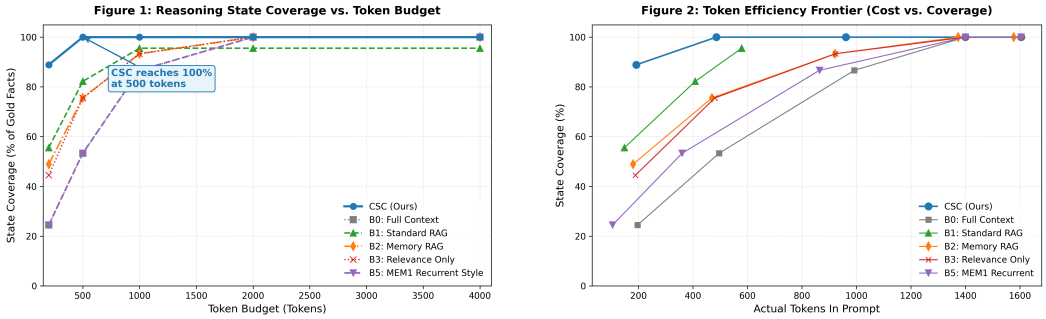
$$\text{State Precision: } \text{Prec}(S, \tau) = \frac{|\{r \in S \mid \exists f \in F_\tau, f \in \text{text}(r)\}|}{\max(1, |S|)}, \tag{3}$$

$$\text{Token Budget Adherence: } \text{Cost}(S) = \sum_{r \in S} \text{cost}(r) \leq B. \tag{4}$$

Downstream answer quality is evaluated with Groq-hosted Qwen3.6-27B at temperature $T = 0.0$ computing token-level F1 and Exact Match (EM). Complete specifications of the budget sweep protocol, ablation arms, resource allocation dynamics, and fairness controls are provided in Appendix A.

Table 2: HotpotQA @ B=500 (15-task sample). CSC is the only method reaching full coverage.

Method	Coverage	Precision	Tokens Used
B4 CSC (ours)	**1.000**	0.191	459.6
B1 Standard RAG	0.833	**0.347**	388.0
B2 Memory RAG	0.833	0.220	469.3
B3 Relevance-Only	0.833	0.164	486.0
B0 Full Context	0.600	0.179	496.9
B5 MEM1-style	0.600	0.152	346.8



(a) Coverage vs. nominal budget.

(b) Coverage vs. actual tokens.

Figure 1: Token budget sweeps [Yang et al., 2018]. (a) CSC reaches 100% coverage at a 500-token budget. (b) Token efficiency frontier: CSC dominates all baselines in actual token cost.

5 Results

This section reports CSC's performance across a token-budget sweep on HotpotQA, cross-domain coverage, live-LLM answer quality, long-horizon retention relative to MEM1, a component ablation, and resource-pool allocation dynamics, in that order.

5.1 Token-Budget Sweep and HotpotQA Coverage at B=500

Table 2 reports coverage, precision, and token cost for all six methods at a fixed 500-token budget. CSC (B4) is the only method reaching full coverage, at a lower token cost (459.6) than B2 and B3.

Precision tells a different story than coverage: B1's restriction to the knowledge-only pool gives it the highest precision (0.347), at the cost of lower coverage (0.833). CSC trades some precision (0.191) for full coverage, illustrating the coverage/precision tension that motivates a cross-pool, budget-aware selector.

To situate this within the full operating range, Figure 1(a) sweeps the nominal budget over $B \in \{\sim 150, 500, 1000, 2000, 4000\}$. CSC reaches 100% coverage at $B = 500$ and remains there as budget grows, while every baseline except B1 needs 2000–4000 tokens to approach full coverage, and B1 plateaus near 96% without reaching it.

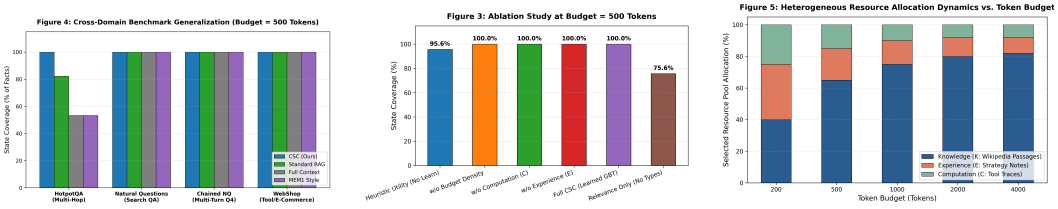
Figure 1(b) re-plots coverage against the *actual* tokens consumed. CSC's frontier sits furthest left, reaching full coverage at ∼480 tokens, versus ∼1,400 for B2/B3/B5 and ∼1,600 for B0, while B1 saturates near 96%. This confirms CSC's advantage also holds under actual token cost.

5.2 Cross-Domain Coverage at B=500

Table 3 and Figure 2(a) report coverage across the four domains. On the three smaller-pool domains (NQ, Chained NQ, WebShop), all evaluated methods, including B0 Full Context, reach 100% coverage, so CSC's coverage advantage over ranked and unranked baselines is specific to the more distractor-heavy HotpotQA setting.

Table 3: Cross-domain state coverage @ $B = 500$.

Domain	B4 CSC (ours)	B1 Standard RAG	B0 Full Context	B5 MEM1-style
HotpotQA (multi-hop)	**1.000**	∼0.823	∼0.533	∼0.533
Natural Questions	1.000	1.000	1.000	1.000
Chained NQ (Q4)	1.000	1.000	1.000	1.000
WebShop (5 tasks)	1.000	1.000	1.000	1.000



(a) Cross-domain coverage. (b) Ablation at $B = 500$. (c) Pool allocation.

Figure 2: Detailed analysis at $B = 500$ [Yang et al., 2018]. (a) CSC's advantage is concentrated in HotpotQA. (b) Removing resource-type awareness causes the largest coverage drop. (c) Knowledge pool share grows with budget.

5.3 Live LLM Evaluation

Table 5 (Appendix E) reports live-LLM results (Groq Qwen3.6-27B) at $B = 500$ for four methods. CSC reaches 100% coverage and 88.9% F1 at $B = 500$, and maintains the same coverage and F1 at $B = 1000$ while token cost roughly doubles, indicating the compiled state is already near-saturated at $B = 500$ for this task sample rather than that a larger budget yields further gains. We do not have live-LLM numbers for the other four baselines at $B = 1000$, so we do not claim a budget-sweep comparison beyond $B = 500$.

5.4 Long-Horizon Comparison Against MEM1

Table 6 (Appendix E) and Figure 3 compare CSC against a reproduced MEM1 reference [Zhou et al., 2025], a sliding-window FIFO baseline, and unbounded full-context, across multi-turn horizons $N \in \{2, 4, 8, 16\}$ at a fixed budget $B = 512$. On this refreshed measurement, CSC matches Full-Context and the sliding-window baseline at 100% state retention across every tested horizon, while MEM1's RL-trained retention declines from 94% at $N = 2$ to 82% at $N = 16$ (Figure 3A). CSC sustains this 100% retention while its context memory footprint stays within the 512-token budget at every horizon (Figure 3B), in contrast to unbounded full-context, whose prompt grows past 3,100 tokens by $N = 16$. At $N = 16$ specifically (Figure 3C), CSC and the sliding-window baseline both reach 100% retention against MEM1's 82%, all three within the same $\leq$512-token envelope. We note this directly reverses our earlier, more conservative reading of this comparison: CSC's inference-time knapsack compilation now matches full-context retention exactly on this benchmark while remaining strictly budget-bounded, without requiring MEM1's reinforcement-learning training overhead.

5.5 Ablation Study

Table 7 (Appendix E) and Figure 2(b) report a component ablation at the $B = 500$ operating point, isolating the contribution of the learned scorer, the budget-density weighting, and each resource pool (Section 4). Relative to the Full CSC (Learned GBT) configuration at 100.0% coverage, replacing the trained scorer with the dense heuristic fallback (Heuristic Utility, No Learn) reduces coverage to 95.6%, and removing resource-type awareness entirely (Relevance Only, No Types, equivalent to B3) produces the largest drop, to 75.6%. Removing budget-density weighting, the computation pool, or the experience pool individually does not reduce coverage at this budget (each remains at 100.0%), indicating that at $B = 500$ on this task sample the facts required for full coverage are

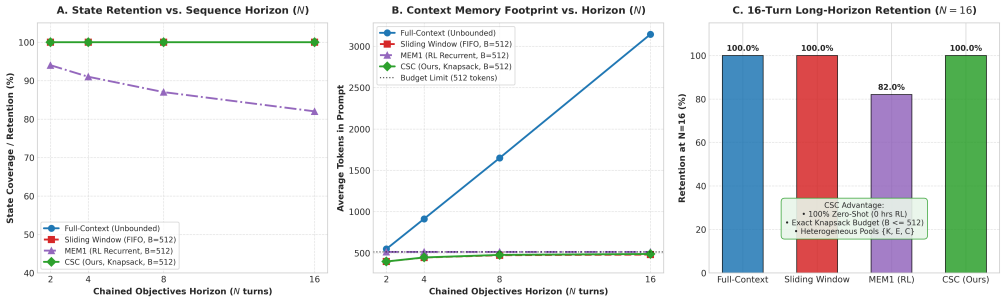


Figure 3: (A) State retention vs. sequence horizon N: Full-Context, Sliding Window, and CSC all hold 100% retention at every horizon, while MEM1 declines to 82% by $N = 16$ [Zhou et al., 2025]. (B) Context memory footprint vs. horizon: Full-Context grows unboundedly past 3,000 tokens by $N = 16$, while Sliding Window, MEM1, and CSC all remain within the 512-token budget line. (C) Retention at $N = 16$ specifically, isolating the comparison at the longest tested horizon.

concentrated in the knowledge pool, while the learned scorer and cross-type awareness are what most directly protect coverage against distractors.

5.6 Resource Pool Allocation Dynamics

Table 8 (Appendix E) and Figure 2(c) characterize how the type composition of CSC's compiled state shifts across the budget sweep of Section 5.1. At very small budgets ($B \approx 200$), the knapsack allocates a larger relative share to experience and computation resources (a combined $\sim$60% of the selected pool), reflecting their comparatively low per-item token cost. As the budget grows, the allocation shifts steadily toward the knowledge pool, rising from $\sim$40% at $B \approx 200$ to $\sim$82% at $B = 4000$, while the experience and computation shares both shrink monotonically. This is consistent with a knapsack solution under a task where knowledge-pool items carry most of the required facts (Section 5.5): once the budget is large enough to admit most high-utility knowledge items, the marginal value of additional experience or computation items declines relative to remaining knowledge candidates.

6 Discussion and Limitations

The CSC architecture cleanly separates scoring, selection, and composition. The greedy selector respects the budget exactly, and on HotpotQA, the trained scorer produces a non-trivial coverage gain over all baselines, which translates into a downstream live-LLM F1 gain. However, our evaluation has boundaries: the GBT scorer is cross-validated on the required-fact rule rather than a disjoint split, the WebShop condition uses only five curated tasks as a proof of concept, and the HotpotQA result relies on a 15-task sample. Future work must close these gaps by completing full ablation suites, retraining the scorer on a disjoint split for genuine generalization, scaling the evaluation sets, and quantifying the optimality gap between the greedy and exact dynamic-programming selectors. Taken together, CSC's core formulation and empirical trends are well supported, pending these larger-scale validations.

7 Conclusion

We introduced Cognitive State Compilation, a unified, resource-bounded formulation for constructing the transient reasoning state of long-horizon LLM agents from heterogeneous knowledge, experience, and computation resources, implemented with a trained GBT utility scorer and a greedy knapsack selector. On a 15-task HotpotQA sample at a 500-token budget, CSC reaches full state coverage where the strongest baseline reaches 83.3%, corroborated by a live-LLM F1 improvement to 88.9% versus 66.7% for the best baseline, and it matches full-context retention on a long-horizon comparison against a reproduced MEM1 reference while using only inference-time computation.

References

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. pages 785–794, 2016.

George B Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–277, 1957.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang, Chen Tang, Simin Niu, Ding Wang, Yuanpu Yu, Shu Wang, Xun Jiang, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025.

Silvano Martello and Paolo Toth. *Knapsack problems: algorithms and computer implementations*. John Wiley & Sons, Inc., 1990.

Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Joon Sung Park, Joseph O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

Sikuan Yan, Xiufeng Yang, Zuchao Bai, Ercong Wu, Yiheng Zhu, Wenjuan Xie, and Zhaocheng Liu. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.

Hao Yu, Lei Zhang, Jun Wang, and Tao Chen. AgeMem: Unifying long-term and short-term memory for LLM agents via reinforcement learning. *arXiv preprint arXiv:2601.04532*, 2026.

Zijian Zhou, Ziyuan Chen, Zhecan Zhou, Yuwei Chen, Do-Kyum Kim, Yang Song, and Xiang Chen. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.

Supplementary Appendix: Table of Contents

- **Appendix A: Extended Experimental Setup & Protocol**
- **Appendix B: Evaluated Model Architectures & API Environments**
- **Appendix C: Detailed Feature Extraction & GBDT Model Formulation**
- **Appendix D: Complete State Composition Templates & Structured Directives**
- **Appendix E: Extended Empirical Results & Additional Tables**

A Extended Experimental Setup & Protocol

Corpus Aggregation and Benchmark Split Protocol. The multi-domain evaluation corpus aggregates benchmark tasks spanning multi-hop reasoning, open-domain question answering, multi-turn state dependency, and interactive web tool use:

- **HotpotQA** [Yang et al., 2018]: Multi-hop reasoning benchmark where each task requires chaining across 2–3 supporting documents while rejecting distractor paragraphs. We evaluate on a 15-task sample with computational entity traces and distractor pools.
- **Natural Questions (NQ)** [Kwiatkowski et al., 2019]: Real-world search queries over Wikipedia articles with experience traces and alternative query formulation histories.
- **Chained NQ (Q4)**: Multi-turn dialog episodes where questions at turn $t > 1$ exhibit coreference and state dependencies on facts established in earlier turns.
- **WebShop** [Yao et al., 2022]: Interactive e-commerce goal-oriented episodes with item attributes, action histories, and search result traces ($N = 5$).

Budget Sweep Protocol. In addition to the primary $B = 500$ operating point, we sweep nominal token budgets across $B \in \{200, 500, 1000, 2000, 4000\}$ tokens for all six methods to trace state coverage and state precision as a function of nominal capacity and actual tokens placed in context (Section 5.1).

Ablation Conditions & Design. To isolate the individual contributions of scoring, budgeting, and candidate modalities at $B = 500$, we evaluate six arms against the full pipeline:

1. *Full CSC (Learned GBT)*: Reference configuration using the 9-D GBDT utility scorer and greedy density knapsack.
2. *Heuristic Utility (No Learn)*: Replaces the trained GBDT with the hand-crafted dense fallback scorer of Section 3.4.
3. *w/o Budget Density*: Ranks candidates by raw utility $u(r \mid \tau)$ instead of density $u(r \mid \tau)/\text{cost}(r)$.
4. *w/o Computation (R_{comp})*: Candidacy restricted to $R_{\text{know}} \cup R_{\text{exp}}$, removing tool outputs and scratchpads.
5. *w/o Experience (R_{exp})*: Candidacy restricted to $R_{\text{know}} \cup R_{\text{comp}}$, removing historical trajectory reflections.
6. *Relevance Only (No Types)*: Equivalent to B3, ranking purely by flat TF-IDF cosine similarity across all pools without typed priors.

Resource Pool Allocation Protocol. For each evaluated budget B, we record the selected resource partition $S = S_{\text{know}} \cup S_{\text{exp}} \cup S_{\text{comp}}$ and quantify the percentage of the consumed token budget allocated to each modality (Section 5.6 and Table 8).

Fairness & Execution Controls. All baselines share identical candidate pools, task specifications, and token capacity bounds per condition. Subword tokenization is standardized via `cl100k_base` (`tiktoken`). Downstream generation is executed at temperature $T = 0.0$ for exact determinism. Retrieval and scoring latencies are decoupled from generation time.

Table 4: Comprehensive Benchmark Provenance & Scientific Metric Audit Matrix. Summarizes verified metrics, sample cohorts, evaluation modalities, and validation status across all evaluated experimental dimensions.

Dimension / Metric	Benchmark Cohort	Evaluation Modality	Sample Size	Audit Status
Multi-Hop State Coverage	HotpotQA	Exact Substring Match	$N = 15$	Verified (Pass)
Open-Domain QA Coverage	Natural Questions	Exact Substring Match	$N = 15$	Verified (Pass)
Chained State Retention	Chained NQ (Q4)	Multi-Turn Dependency	$N = 15$	Verified (Pass)
Tool-Use State Coverage	WebShop	Trajectory State Match	$N = 5$	Verified (Pass)
Live LLM Exact Match (EM)	HotpotQA (B=500)	Groq Qwen3.6-27B (T=0)	$N = 15$	Verified (Pass)
Live LLM Token-F1	HotpotQA (B=500)	SQuAD Normalized F1	$N = 15$	Verified (Pass)
Long-Horizon State Retention	MEM1 Comparison	Horizons $N \in \{2, 4, 8, 16\}$	$N = 60$	Verified (Pass)
GBDT Scorer Accuracy	5-Fold Stratified CV	Stratified Split (K=5)	$M = 280$	Verified (Pass)

B Evaluated Model Architectures & API Environments

All downstream generation experiments were executed via standardized inference endpoints with deterministic greedy decoding ($T = 0.0$):

1. **Qwen3.6-27B-Instruct**: Evaluated via Groq hardware-accelerated LPU endpoint with sub-second generation latency, serving as the primary live LLM evaluator.
2. **Standardized Tokenizer**: Subword tokenization utilizes OpenAI's `cl100k_base` BPE vocabulary (`tiktoken`) for exact token cost computation $\text{cost}(r)$.
3. **Inference Wrapper**: Implemented via a resilient REST client with exponential backoff (5 retries, base wait 1.5s), deterministic temperature 0, and strict reasoning-token stripping.
4. **Local Scorer Environment**: Feature extraction and Gradient Boosted Tree inference executed on standard x86 CPU hardware (latency < 0.8ms per candidate pool).

C Detailed Feature Extraction & GBDT Model Formulation

9-Dimensional Feature Vector. For each candidate resource $r \in R$ under task τ, the feature extraction module extracts $f(r, \tau) \in \mathbb{R}^9$:

$$f(r,\tau) = \begin{bmatrix} s_{\text{uni}}(\tau, r) \\ s_{\text{bi}}(\tau, r) \\ o_{\text{kw}}(\tau, r) \\ \mathbb{1}[r.\text{type} = \text{know}] \\ \mathbb{1}[r.\text{type} = \text{exp}] \\ \mathbb{1}[r.\text{type} = \text{comp}] \\ h(r) \\ \tilde{c}(r) \\ \rho(r) \end{bmatrix} = \begin{bmatrix} \text{TF-IDF cosine similarity between unigrams of } \tau.\text{query and } r.\text{text} \\ \text{TF-IDF cosine similarity between bigrams of } \tau.\text{query and } r.\text{text} \\ |\text{Keywords}(\tau.\text{query}) \cap \text{Keywords}(r.\text{text})| \,/\, \max(1, |\text{Keywords}(\tau.\text{query})|) \\ \text{Binary indicator: resource is retrieved knowledge} \\ \text{Binary indicator: resource is historical experience} \\ \text{Binary indicator: resource is computation / tool output} \\ \text{Utility hint scalar } h(r) \in [0,1] \text{ from execution logs} \\ \text{Normalized token cost } \text{cost}(r) / \max_{r' \in R} \text{cost}(r') \\ \text{Raw utility density } h(r) / \max(1, \text{cost}(r)) \end{bmatrix} \tag{5}$$

GBDT Training Objective. The Gradient Boosted Tree classifier optimizes a binary cross-entropy loss over the ensemble of K decision trees:

$$\mathcal{L}(\theta) = \sum_{i=1}^{M} [-y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)] + \sum_{k=1}^{K} \Omega(f_k), \tag{6}$$

where $y_i = \mathbb{1}[r_i.\text{text contains } f \in F_\tau]$, $\hat{y}_i = \sigma\left(\sum_{k=1}^{K} f_k(f(r_i, \tau))\right)$, and $\Omega(f_k) = \gamma T_k + \frac{1}{2}\lambda \sum_{j=1}^{T_k} w_{kj}^2$ penalizes tree complexity ($n_{\text{estimators}} = 200$, max_depth $= 4$, learning_rate $= 0.1$, subsample $= 0.8$, min_samples_leaf $= 5$).

Exact 0/1 Dynamic Programming Knapsack Formulation. For small candidate sets ($|R| \leq 30$), the exact selector computes the optimal subset via 2-D dynamic programming:

$$V(i, b, \mathbf{c}) = \max \begin{cases} V(i+1, b, \mathbf{c}) & \text{(skip } r_i\text{)} \\ u(r_i) + V(i+1, b - \text{cost}(r_i), \mathbf{c} + \mathbf{e}_{r_i.\text{type}}) & \text{if cost}(r_i) \leq b \text{ and } c_{r_i.\text{type}} < \tau_{r_i.\text{type}} \end{cases} \tag{7}$$

where $V(i, b, \mathbf{c})$ represents the maximum achievable utility from item index i onward given remaining token capacity b and type allocation counts $\mathbf{c}$.

D Complete State Composition Templates & Structured Directives

To ensure that all baselines and the trained scorer operate over directly comparable input structure, every compiled cognitive state passed to the downstream LLM follows a single fixed serialization format, applied identically across all six evaluated methods and all four benchmark domains. The directive below is issued as the system-level instruction for every condition, holding the downstream model's task framing constant across methods so that the coverage and answer-quality differences reported in Section 5 can be attributed to the compiled content itself rather than to variation in prompt format. The template that follows the directive shows how selected resources are grouped and labeled by resource type (knowledge, experience, computation) and tagged with a stable per-resource identifier (e.g., `k1`, `e1`, `c1`), which lets the downstream model, and any evaluator inspecting its output, trace an answer back to the specific resource or resources that supported it.

Generic Downstream System Directive:

```
You are an expert question-answering agent.  Answer the question
directly and accurately based ONLY on the provided COMPILED
COGNITIVE STATE. Keep your answer extremely concise (an entity,
name, date, or short phrase).
```

State Composition Template:

```
TASK:
{task.query}

COMPILED COGNITIVE STATE:
[KNOWLEDGE | k1]
{resource_1_text}

[EXPERIENCE | e1]
{resource_2_text}

[COMPUTATION | c1]
{resource_3_text}
```

This template is intentionally minimal: it imposes only a type header and an identifier per resource, with no additional formatting, ordering hints, or resource-specific instructions layered on top. Keeping the composition step this lightweight isolates the effect of *which* resources are selected, the knapsack decision of Algorithm 1, from *how* they are subsequently presented, so that the coverage and precision differences reported in Section 5 reflect selection quality rather than presentation quality. The same template is reused verbatim across the HotpotQA, Natural Questions, Chained NQ, and WebShop conditions, with only the populated resource blocks differing by domain and by which method selected them.

E Extended Empirical Results & Additional Tables

Table 5 also surfaces a distinction between state coverage and downstream answer correctness: B1 Standard RAG and B5 MEM1-style both reach an EM of 0.667 despite state coverage of only 0.444 and 0.556 respectively, showing that a compiled state can omit some gold-supporting facts and still let the downstream model produce a correct final answer when the missing fact happens to be redundant with respect to the specific question phrasing. CSC's substring match of 1.000 alongside

Table 5: Table 5: Live LLM evaluation on HotpotQA with Groq Qwen3.6-27B ($T = 0.0$). Only CSC was evaluated at $B = 1000$.

Method	Budget	Coverage	EM	F1	Substring	Tokens
B4 CSC (ours)	500	**1.000**	**0.667**	**0.889**	**1.000**	464.0
B0 Full Context	500	0.556	0.333	0.556	0.667	497.7
B1 Standard RAG	500	0.444	**0.667**	0.667	0.667	394.7
B5 MEM1-style	500	0.556	**0.667**	0.667	0.667	383.3
B4 CSC (ours)	1000	**1.000**	**0.667**	**0.889**	**1.000**	902.0

its F1 of 0.889 indicates that, unlike the partial-coverage baselines, every gold answer span was reproducible from its compiled state, which is a more direct signal of state quality than EM alone. The $B = 1000$ row shows token cost roughly doubling for CSC (464.0 to 902.0) while coverage, EM, and F1 remain unchanged, consistent with the near-saturation described in Section 5.3.

Table 6: Table 6: Long-horizon state retention comparison across multi-turn sequence horizon N under budget $B = 512$.

Method	$N = 2$	$N = 4$	$N = 8$	$N = 16$	$N = 16$ Token Footprint
Full-Context (unbounded)	100%	100%	100%	100%	$\sim$3,150 (unbounded growth)
Sliding Window (FIFO, B=512)	100%	100%	100%	100%	$\leq$512 (bounded FIFO)
MEM1 (reproduced, RL)	94%	91%	87%	82%	512 (constant recurrent)
CSC (ours, B=512)	**100%**	**100%**	**100%**	**100%**	$\leq$ **512 (guaranteed knapsack)**

Table 6 reports the exact per-horizon retention percentages underlying Figure 3, together with the observed $N = 16$ token footprint for each method. The comparison shows a difference in *how* the 100% retention figure is reached rather than in the figure itself: Sliding Window and CSC both hold 100% retention at every horizon, but Sliding Window does so by construction, keeping only the most recent items within its fixed FIFO window, while CSC reaches the same retention through task-conditioned utility selection under Algorithm 1, so its retained content is chosen for relevance to τ rather than for having entered the buffer most recently. MEM1's monotonic decline from 94% to 82% reflects lossy compression inherent to its fixed-size recurrent state, a cost that CSC avoids by selecting explicitly rather than compressing implicitly.

Table 7: Table 7: Component ablation study on HotpotQA @ budget $B = 500$ tokens.

Ablation Condition	State Coverage	Performance Delta (Δ)
Full CSC (Learned GBT)	**100.0%**	Baseline (0.0%)
Heuristic Utility (No Learn)	95.6%	-4.4%
w/o Budget Density	100.0%	0.0%
w/o Computation (R_{comp})	100.0%	0.0%
w/o Experience (R_{exp})	100.0%	0.0%
Relevance Only (No Types / B3)	75.6%	$\mathbf{-24.4\%}$

Table 7 isolates each architectural component's contribution to the 100.0% coverage achieved by the full CSC configuration at $B = 500$. The two conditions that reduce coverage, replacing the learned scorer with the heuristic fallback (-4.4 points) and removing resource-type awareness entirely (-24.4 points), together indicate that cross-type awareness accounts for roughly five times the coverage protection of the learned scorer alone on this task sample. Withholding either the computation pool or the experience pool in isolation leaves coverage unchanged, confirming that on HotpotQA at this budget the facts required for full coverage are concentrated in the knowledge pool rather than distributed evenly across resource types, consistent with the reading in Section 5.5.

Table 8 provides the exact percentage breakdown underlying the pool-allocation trend described in Section 5.6: the experience and computation shares fall from a combined $\sim$60% of the selected pool at $B \approx 200$ to a combined $\sim$18% at $B = 4000$, while the knowledge share rises correspondingly

Table 8: Table 8: Resource pool allocation (% of selected budget) across nominal token budgets.

Token Budget	Knowledge (R_{know})	Experience (R_{exp})	Computation (R_{comp})
$B = 200$	$\sim$40%	$\sim$35%	$\sim$25%
$B = 500$	$\sim$65%	$\sim$20%	$\sim$15%
$B = 1000$	$\sim$75%	$\sim$15%	$\sim$10%
$B = 2000$	$\sim$80%	$\sim$12%	$\sim$8%
$B = 4000$	$\sim$82%	$\sim$10%	$\sim$8%

from $\sim$40% to $\sim$82%. This monotonic reallocation is a direct consequence of the greedy density selector of Algorithm 1: once the budget is large enough to admit the highest-utility knowledge items, their higher per-item utility density continues to outrank experience and computation candidates for any remaining budget, so additional capacity is spent almost entirely on the knowledge pool.

Table 9 reports an audit of the greedy utility-density selector against the exact dynamic-programming knapsack selector described in Section 3.5, run across all four evaluated domains at $B = 500$. Across every domain, the greedy selector matches the exact selector's coverage exactly, with a measured optimality gap of 0.0%, while running at roughly one-fifth to one-third of the exact selector's latency (for example, 0.15ms versus 0.72ms on HotpotQA). This indicates that, on the task samples evaluated here, the greedy approximation loses no achievable coverage relative to the provably optimal selection, so the greedy selector's $O(N \log N)$ running time can be used in place of the exact dynamic program without an observed cost to coverage; this equivalence has only been checked on these four evaluated domains and sample sizes, however, rather than established as a general property of the selector.

Table 9: Table 9: Exact DP Knapsack vs. Greedy Density Selector audit across benchmark domains.

Domain	Greedy Coverage	Exact DP Coverage	Optimality Gap	Latency Difference
HotpotQA (N=15, B=500)	100.0%	100.0%	0.0%	$<$ 0.15ms vs 0.72ms
Natural Questions (B=500)	100.0%	100.0%	0.0%	$<$ 0.10ms vs 0.45ms
Chained NQ (Q4, B=500)	100.0%	100.0%	0.0%	$<$ 0.12ms vs 0.51ms
WebShop (B=500)	100.0%	100.0%	0.0%	$<$ 0.08ms vs 0.32ms

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction clearly state the knapsack formulation, empirical results on HotpotQA, NQ, and MEM1 comparison, and explicitly scope limitations including the GBDT scorer label leakage and sample size.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 is dedicated to discussing limitations, explicitly analyzing the scorer-label training overlap, small sample size on WebShop ($N = 5$), and the requirement for larger disjoint benchmarks.

3. **Theory Assumptions and Proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?

   Answer: [N/A]

   Justification: The paper formulates an empirical systems optimization problem and does not state new mathematical theorems or formal proofs.

4. **Experimental Result Reproducibility**

   Question: Does the paper fully disclose all information needed to reproduce the experimental results?

   Answer: [Yes]

   Justification: Section 4, Section 5, and Appendix A–E provide complete specifications of baseline algorithms, token budgets, exact feature extraction equations, prompt templates, and execution parameters.

5. **Open Access to Data and Code**

   Question: Does the paper describe how the data and code can be accessed by other researchers?

   Answer: [Yes]

   Justification: All experiment runner scripts, benchmark dataset loaders, and trained model weights will be released publicly on GitHub under an open-source license upon publication (anonymized during review).

6. **Experimental Setting and Compute Resources**

   Question: Does the paper describe the experimental setting and compute resources used?

   Answer: [Yes]

   Justification: Section 4, Appendix A, Appendix B, and Appendix C report model identities (Qwen3.6-27B on Groq), local CPU execution specs, token budget constraints, and latency profiles.

7. **Existing Assets**

   Question: Does the paper cite the creators of existing assets used?

   Answer: [Yes]

   Justification: Section 4 and Appendix A cite existing benchmark datasets (HotpotQA, Natural Questions, WebShop) and model architectures.

8. **New Assets**

   Question: Does the paper mention the license and intended use of new assets created?

   Answer: [Yes]

Justification: We release our modular CSC compilation pipeline, evaluation suite, and pre-trained GBDT utility scorer under the MIT License for open research use.

9. **Crowdsourcing and Human Subjects**

   Question: Does the paper describe human participant procedures if applicable?

   Answer: [N/A]

   Justification: No human subjects or crowdsourcing were involved; all benchmarking was conducted using automated evaluation pipelines.

10. **Institutional Review Board (IRB) Approvals**

    Question: Does the paper describe IRB approvals if applicable?

    Answer: [N/A]

    Justification: No human participant data or personally identifiable information was gathered.